

PART 3: COMBINATIONAL LOGIC ANALYSIS AND DESIGN PROCEDURES

SAAD BAIG – DIGITAL LOGIC DESIGN

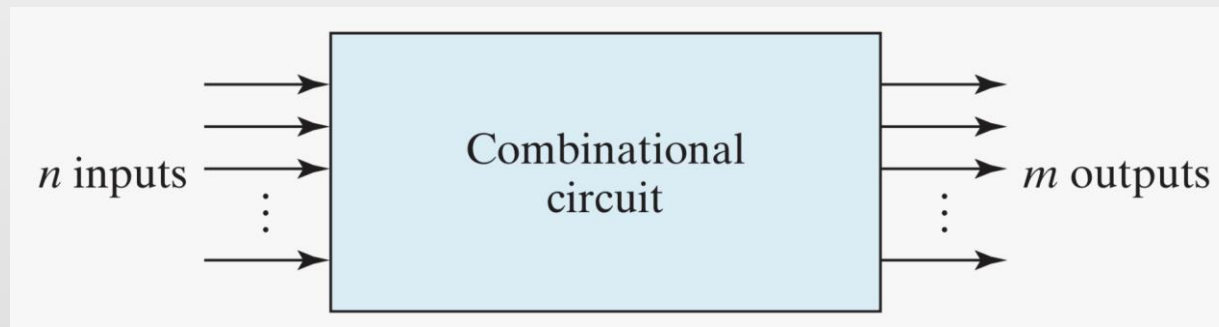


The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from <https://www.hasanbaig.com/resources>

© Hasan Baig and Junaid Ahmed Memon. All rights reserved. These files and accompanying materials are made available under the terms of “Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) License” and is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>

WHAT IS COMBINATIONAL LOGIC?

- Outputs at any time are determined from only the present combination of inputs.
- A combinational circuit consists of input variables, logic gates, and output variables.



- For n input variables, there are 2^n possible binary input combinations.
- For each possible input combination, there is one possible output value.



WHAT IS COMBINATIONAL LOGIC?

- A combinational circuit can be specified by a truth table.
 - **Example:** The following truth table describes a combinational circuit with $n = 2$, and $m = 2$.
- A combination circuit also can be described by m Boolean functions.
- Total possible functions = 2^{2^n}
 - $f_1 = x_1'x_2'$
 - $f_2 = x_1x_2'$

x_1	x_2	f_1	f_2
0	0	1	0
0	1	0	0
1	0	0	1
1	1	0	0



ANALYSIS PROCEDURE

- The analysis of a combinational circuit requires that we determine the function that the circuit implements
- The analysis task starts with a given logic diagram
- Obtain a set of Boolean functions, a truth table, or, possibly, an explanation of the circuit operation
- The first step is to make sure that the given circuit is combinational and not sequential.

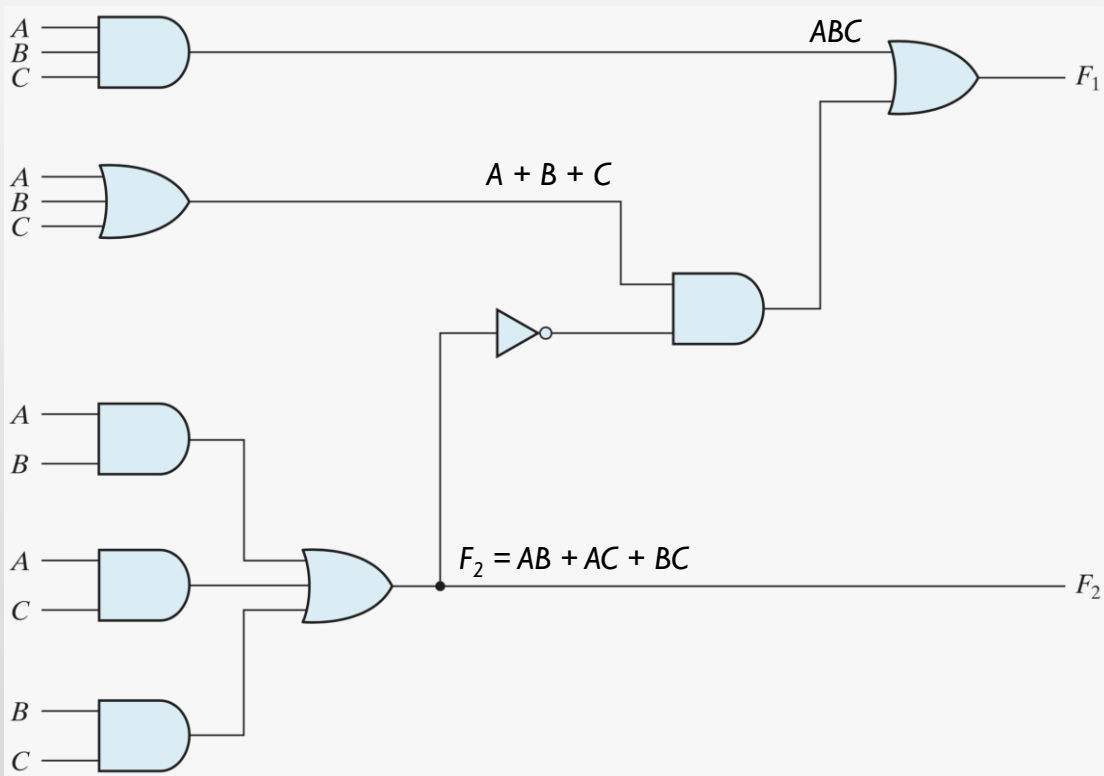
The diagram of a combinational circuit has logic gates with no feedback paths or memory elements. A circuit with feedback or memory is sequential.



ANALYSIS: LOGIC DIAGRAM TO BOOLEAN FUNCTION

1. Label all gate outputs that are a function of input variables with arbitrary symbols— but with meaningful names. Determine the Boolean functions for each gate output.
2. Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols. Find the Boolean functions for these gates.
3. Repeat the process outlined in step 2 until the outputs of the circuit are obtained.
4. By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of input variables.

ANALYSIS: LOGIC DIAGRAM TO BOOLEAN FUNCTION



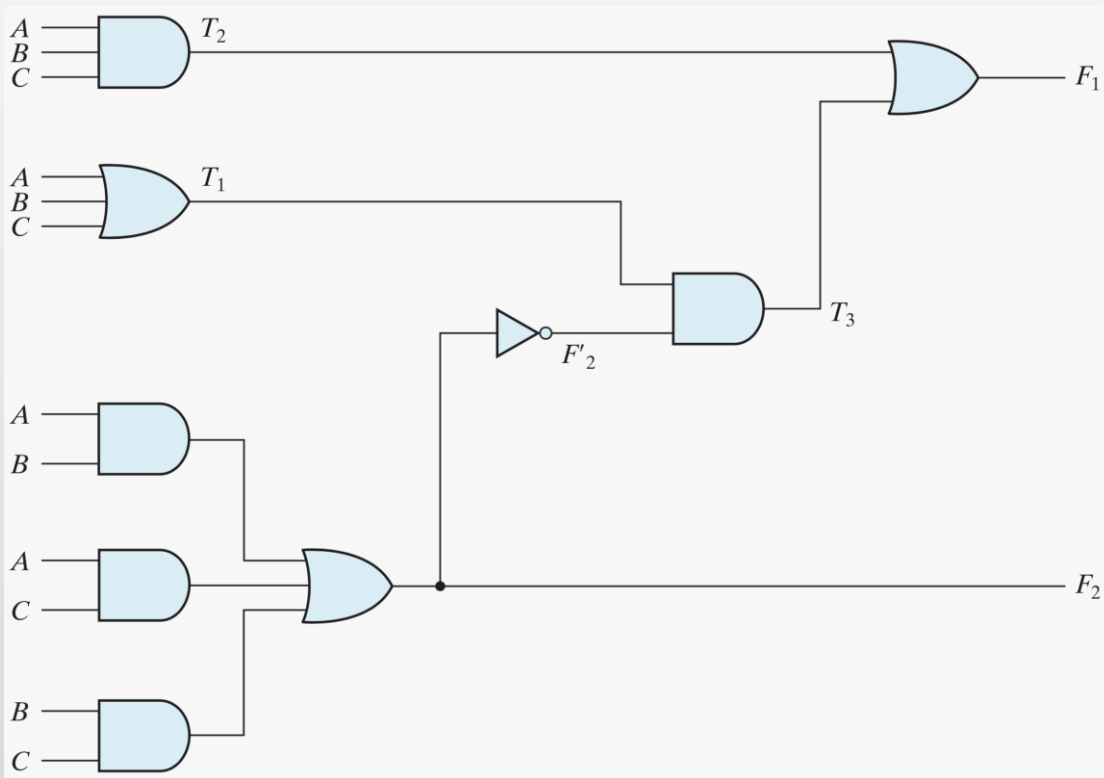
- $F_1 = ABC + F_2'(A + B + C)$
- $F_1 = ABC + (AB + AC + BC)'(A + B + C)$
- $F_1 = ABC + (AB)'(AC)'(BC)'(A + B + C)$
- $F_1 = ABC + (A' + B')(A' + C')(B' + C')(A + B + C)$
- $F_1 = ABC + (A' + A'C' + A'B' + B'C')(AB' + B'C + AC' + BC')$
- $F_1 = ABC + (A'(1 + C' + B') + B'C')(AB' + B'C + AC' + BC')$
- $F_1 = ABC + (A' + B'C')(AB' + B'C + AC' + BC')$
- $F_1 = ABC + A'B'C + A'BC' + AB'C'$



ANALYSIS: LOGIC DIAGRAM TO TRUTH TABLE

1. Determine the number of input variables in the circuit. For n inputs, form the 2^n possible input combinations
2. Label the outputs of selected gates with arbitrary symbols.
3. Obtain the truth table for the outputs of those gates which are a function of the input variables only.
4. Proceed to obtain the truth table for the outputs of those gates which are a function of previously defined values until the columns for all outputs are determined.

ANALYSIS: LOGIC DIAGRAM TO TRUTH TABLE



A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

DESIGN PROCEDURE



- The design of a combinational circuit starts from the (verbal) specification of the design objective and culminates in a logic circuit diagram or a set of Boolean functions.
- Verbal specifications should be interpreted correctly by deriving the truth table, as they are often incomplete. Any wrong interpretation may result in an incorrect design.



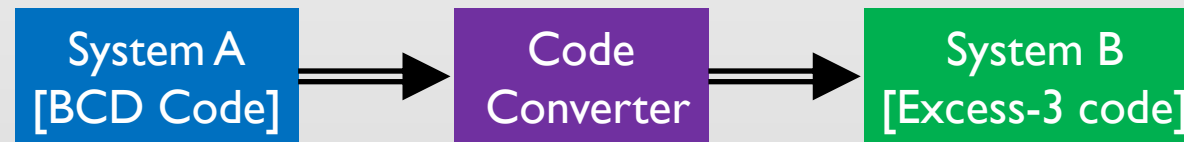
DESIGN: LOGIC DIAGRAM FROM SPECIFICATIONS

1. From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each.
2. Derive the truth table that defines the required relationship between inputs and outputs.
3. Obtain the simplified Boolean functions for each output as a function of the input variables.
4. Draw the logic diagram and verify the correctness of the design (manually or by simulation).



DESIGN ACTIVITY: CODE CONVERSION

- Given two systems i.e. System A and System B. System A uses BCD code and System B uses Excess-3 code to exchange information i.e. 0-9 numbers. Design a code converter circuit which receives inputs from system A in BCD code and gives equivalent output to system B in excess-3 code.



- A **code converter** is a circuit that makes the two systems compatible even though each uses a different binary code.



DESIGN ACTIVITY: CODE CONVERSION

- I. From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each.

- Number of inputs = 04
- Number of outputs = 04



Decimal Digit	BCD 8421	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100



DESIGN ACTIVITY: CODE CONVERSION

2. Derive the truth table that defines the required relationship between inputs and outputs.



Decimal Digit	BCD 8421	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

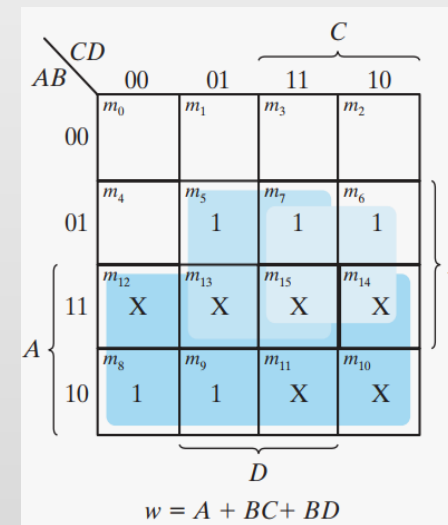
Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

DESIGN ACTIVITY: CODE CONVERSION

- Obtain the simplified Boolean functions for each output as a function of the input variables.



Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

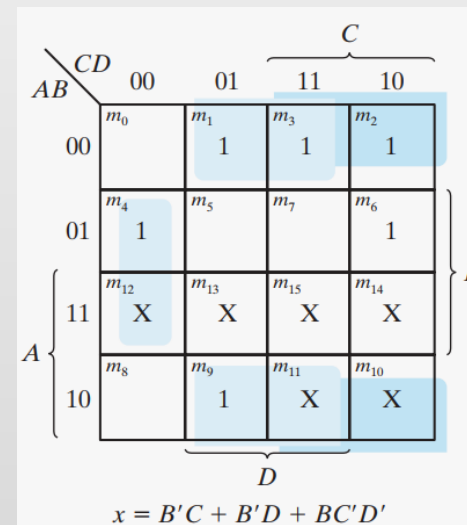


DESIGN ACTIVITY: CODE CONVERSION

- Obtain the simplified Boolean functions for each output as a function of the input variables.



Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

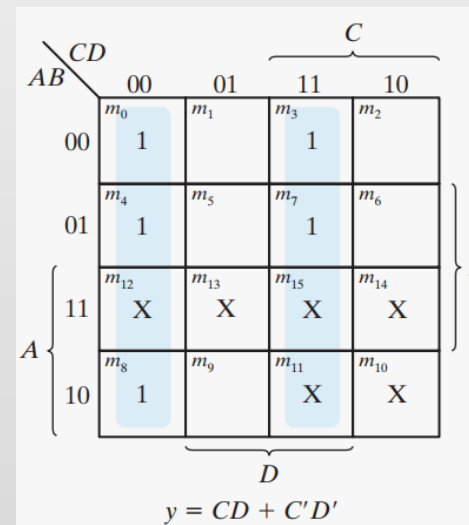


DESIGN ACTIVITY: CODE CONVERSION

- Obtain the simplified Boolean functions for each output as a function of the input variables.



Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

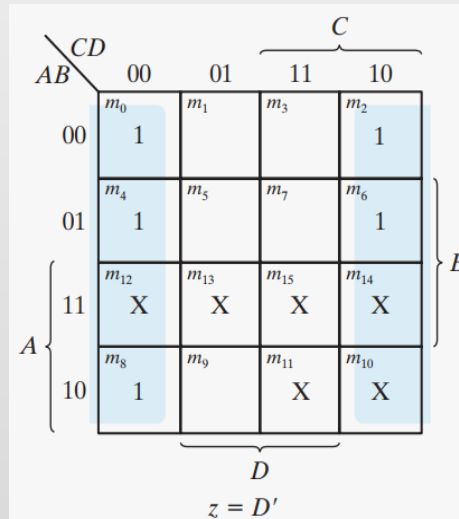


DESIGN ACTIVITY: CODE CONVERSION

- Obtain the simplified Boolean functions for each output as a function of the input variables.

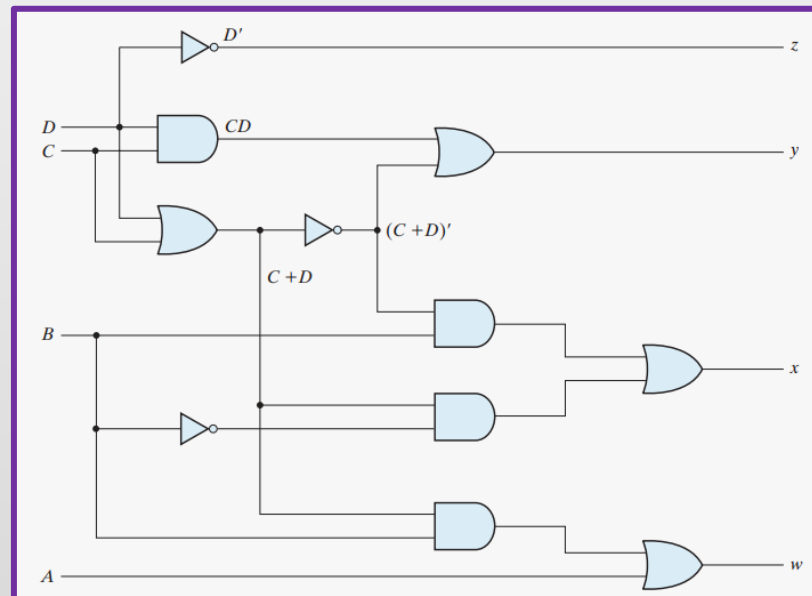


Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0



DESIGN ACTIVITY: CODE CONVERSION

4. Draw the logic diagram and verify the correctness of the design (manually or by simulation).



$$\begin{aligned}
 z &= D' \\
 y &= CD + C'D' = CD + (C + D)' \\
 x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\
 &= B'(C + D) + B(C + D)' \\
 w &= A + BC + BD = A + B(C + D)
 \end{aligned}$$